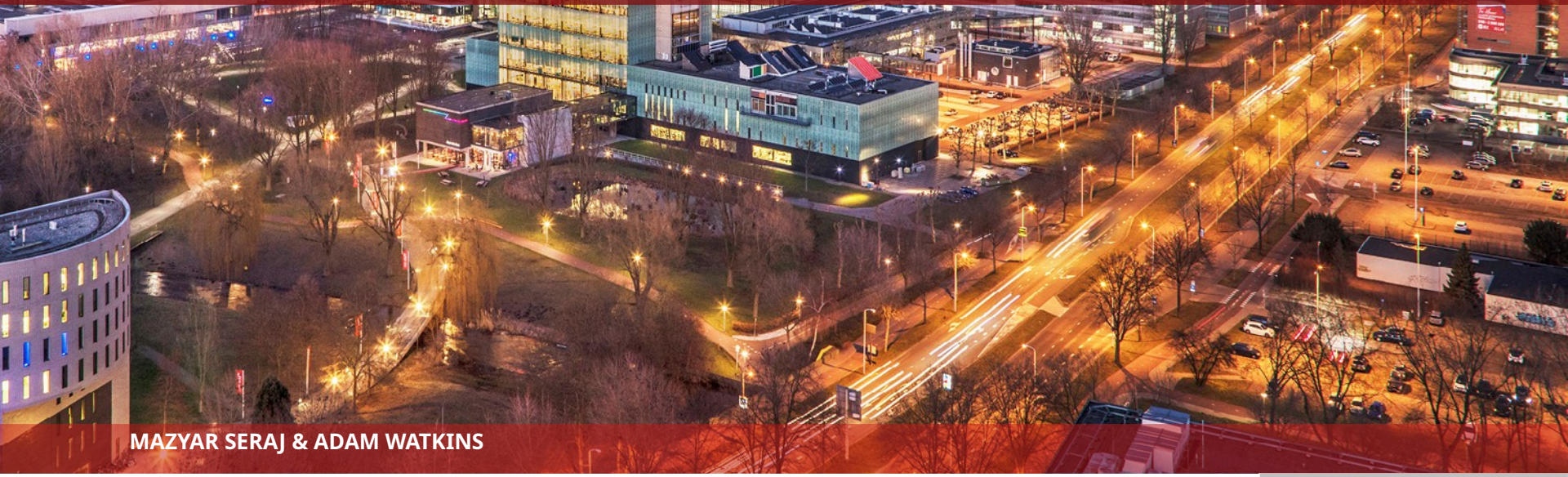
2IRR10 - Autonomous Systems Twinning

TROUBLESHOOTING TURTLEBOT & ROS 2 JAZZY | ADVANCED ROS 2 TOPICS



MAZYAR SERAJ & ADAM WATKINS

M&CS Department – SET & IRIS Clusters

TU/e EINDHOVEN UNIVERSITY OF TECHNOLOGY

Where this lecture fits

Lecture W1 – Diagramming

Context diagram, concept map, Quality indicators

Lecture W2 – ROS2 Basics

nodes, topics, services, actions, RViz, Gazebo

Lecture W3 – Smart Waiter Example

Example of Smart Waiter:
Problem → Solution

Lecture W6 – Troubleshooting

diagnose failures in robot / ROS / twin system layer by layer
QoS, actions, params, rosbag2, TF2, SLAM/Nav2 overview

Learning outcome!

By the end of this lecture, you should know how to diagnose failures syste

1. **Locate the fault domain:**
   hardware, ROS graph, TF, RViz, Gazebo, network
2. **Use ROS 2 CLI tools:**
   node / topic / service / action / param inspection
3. **Debug twin-specific failures:**
   bidirectional flow, state sync, latency, environment events
4. **Twin system becomes distributed and robust:**
   Launch files, TF2, Nav2, QoS, rosbag2
5. **Prepare for the final lab review in week 8:**
   evidence: logs, topic echoes, screenshots, demos

Learning outcome!

By the end of this lecture, you should know how to diagnose failures syste

1. **Locate the fault domain:**
   hardware, ROS graph, TF, RViz, Gazebo, network
2. **Use ROS 2 CLI tools:**
   node / topic / service / action / param inspection
3. **Debug twin-specific failures:**
   bidirectional flow, state sync, latency, environment events
4. **Twin system becomes distributed and robust:**
   Launch files, TF2, Nav2, QoS, rosbag2
5. **Prepare for the final lab review in week 8:**
   evidence: logs, topic echoes, screenshots, demos

Troubleshooting mindset

Do not debug randomly.
Move from symptom → evidence → hypothesis → test

1

Symptom
What exactly is wrong?

2

Evidence
Which command or UI proves it?

3

Hypothesis
Which layer might be failing?

4

Test
Run one focused check

5

Fix / Rollback
Apply change or return to last working version

Where can a TurtleBot twin fail?

Separate the system into fault domains before choosing a diagnostic tool.

Hardware

battery, wheels, LiDAR, OpenCR firmware, SBC

Ros Graph

nodes, topics, services, actions, parameters

Frames / TF

map, odom, base_link, laser relationships

Visualization

RViz fixed frame, displays, QoS mismatch, sim_time

Simulation

Gazebo model/world, bridges

Network

ROS_DOMAIN_ID, Wi-Fi, IP, DDS discovery

A practical diagnostic pipeline

Use this order when a robot, simulation, or twin does not behave as expected.

1 Is the node running?

```
ros2 node list
ros2 node info /node_name
```

2 Does the topic exist?

```
ros2 topic list
ros2 topic info /node_name -v
```

3 Is data arriving?

```
ros2 topic echo /scan
ros2 topic hz /odom
```

4 Are frames valid?

```
ros2 run tf2_tools
view_frames
rviz2 TF display
```

5 Does RViz show it?

```
Fixed Frame + displays
LaserScan / Odometry
```

6 Does Gazebo match?

```
ros_gz bridge active?
use_sim_time / clock
```

TU/e

ROS 2 CLI quick reference

CLI = the fastest way to inspect the ROS graph without launching any UI

Nodes

```
ros2 node list
ros2 node info
/node_name
```

Services

```
ros2 service list
ros2 service type
/srv
ros2 service call
/srv ...
```

Actions

```
ros2 action list
ros2 action info /act
ros2 action send_goal
...
```

Topics

```
ros2 topic list
ros2 topic info /odom
-v
ros2 topic echo /scan
ros2 topic hz /odom
```

Parameters

```
ros2 param list
ros2 param get /node
p
ros2 param set /node
p v
```

Common TurtleBot symptoms and first checks

Start with the symptoms you actually encounter in the lab

Robot does not move	Is /cmd_vel arriving? Battery ok? turtlebot3_node running on SBC?
LiDAR missing on /scan	LDS driver launched? USB port permissions? TURTLEBOT3_MODEL set?
RViz shows errors / no data	Correct Fixed Frame? Is /tf publishing? use_sim_time consistent?
Sim moves, real robot does not	Wrong ROS_DOMAIN_ID? Missing robot bringup? Topic namespace clash?
Twin desynchronizes	QoS mismatch? Topic namespace collision? Bridge not running?
Navigation fails or loops	Map loaded? AMCL/SLAM active? Costmap footprint? Goal tolerance?

Common TurtleBot symptoms and first checks

Start with the symptoms you actually encounter in the lab

Robot does not move	Is /cmd_vel arriving? Battery ok? turtlebot3_node running on SBC?
LiDAR missing on /scan	LDS driver launched? USB port permissions? TURTLEBOT3_MODEL set?
RViz shows errors / no data	Correct Fixed Frame? Is /tf publishing? use_sim_time consistent?
Sim moves, real robot does not	Wrong ROS_DOMAIN_ID? Missing robot bringup? Topic namespace clash?
Twin desynchronizes	QoS mismatch? Topic namespace collision? Bridge not running?
Navigation fails or loops	Map loaded? AMCL/SLAM active? Costmap footprint? Goal tolerance?

Twin desynchronizes

Why 3 m in Gazebo ≠ 3 m on the real robot?
Why 90 degree in Gazebo ≠ 90 degree on the real robot?

Gazebo uses ideal physics, while the real robot has mechanical, electrical, and environmental imperfections.

Gazebo (ideal)

- Perfect wheel radius (no wear)
- No floor friction or wheel slip
- No motor backlash or gearbox play
- Battery voltage constant
- Encoder resolution infinite

→ /odom exactly matches real position

Real TurtleBot3 Burger

- Mechanical: Wrong wheel_radius or wheel_separation in firmware
- Slip: Floor surface causes wheels to slip (tile, smooth floor)
- Backlash: DYNAMIXEL gearbox play at low speeds
- Electrical: Battery voltage drop changes motor torque per run
- Asymmetry: Left ≠ right wheel diameter → rotation bias

Command 3 m → Gazebo moves exactly 3 m.
Real robot may move 2.7 m or 3.2 m depending on floor, battery level, and wheel wear.

This gap between /odom reported and actual position IS the sync error your DT must measure and log → it directly motivates real-time state synchronization.

Your options - choose based on your POC

There is no single correct answer. Pick the approach that fits what your project is trying to do

Calibrate wheel_radius and wheel_separation

When: Short distances, no map, acceptable small error

```
ros2 param set /turtlebot3_node wheel_radius <value>
ros2 param set /turtlebot3_node wheel_separation <value>
```

Use the SLAM map and use Nav2 for goal navigation

When: Repeatable, accurate navigation over longer distances

SLAM Toolbox builds the map; Nav2 uses LiDAR to correct pose continuously — wheel drift becomes irrelevant.

Accept the drift and measure it

When: If the sync gap itself is your twin evidence

Log /odom on both sides simultaneously. The difference between Gazebo and real robot /odom is your measured sync error.

TU/e

Fix: calibrate wheel radius and wheelbase

Necessary commands can be found here!

The two firmware parameters with the biggest impact on linear and rotational accuracy

Key parameters (turtlebot3_burger.yaml on OpenCR / SBC)

```
wheel_radius: 0.033        # metres — measure your actual wheel
wheel_separation: 0.160    # metres — measure centre-to-centre
```

Calibrate wheel_radius (distance)

- Mark a straight 3 m line on the floor
- Command exactly 3 m: ros2 topic pub /cmd_vel ...
- Measure actual distance travelled
- Scale: new_radius = old × (actual / commanded)
- Repeat until error < 1%

Calibrate wheel_separation (rotation)

- Command a full 360° rotation in place
- Measure actual rotation with a protractor or /imu
- Scale: new_separation = old × (commanded / actual)
- Repeat until rotation error < 2°
- Apply both fixes together before re-testing

Example:

```
ros2 param set /turtlebot3_node wheel_radius 0.0335
ros2 param set /turtlebot3_node wheel_separation 0.1615
```

TU/e

Common TurtleBot symptoms and first checks

Start with the symptoms you actually encounter in the lab

Robot does not move	Is /cmd_vel arriving? Battery ok? turtlebot3_node running on SBC?
LiDAR missing on /scan	LDS driver launched? USB port permissions? TURTLEBOT3_MODEL set?
RViz shows errors / no data	Correct Fixed Frame? Is /tf publishing? use_sim_time consistent?
Sim moves, real robot does not	Wrong ROS_DOMAIN_ID? Missing robot bringup? Topic namespace clash?
Twin desynchronizes	QoS mismatch? Topic namespace collision? Bridge not running?
Navigation fails or loops	Map loaded? AMCL/SLAM active? Costmap footprint? Goal tolerance?

TU/e

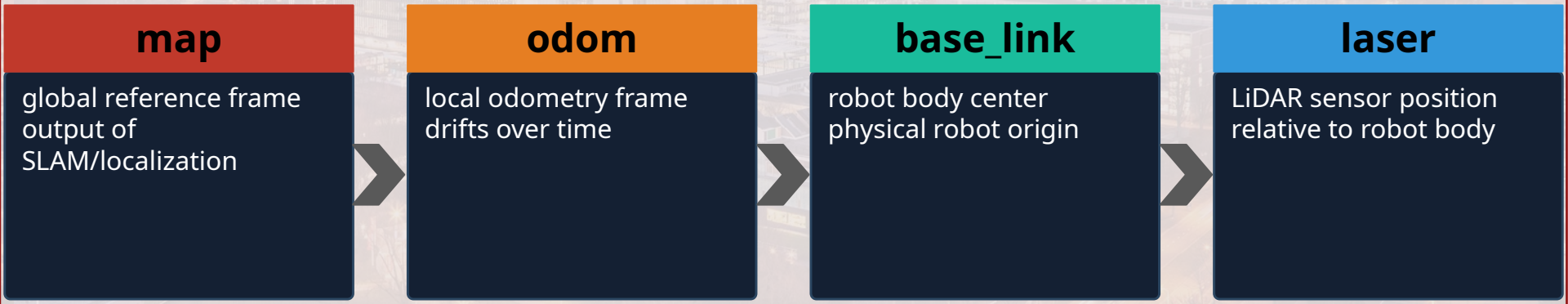
Gazebo troubleshooting: simulation is another system

Check model, world, bridge, topics, and clock - not just the rendered picture.

Robot visible but not moving	Check /cmd_vel message type and ros_gz_bridge mapping
Sensors visible, no ROS topic	Check ros_gz_bridge configuration and topic name spelling
RViz data delayed or wrong	Check /clock topic and use_sim_time flag on all nodes
Collision or stuck robot	Check world geometry, friction coefficients, spawn pose
Real and sim topic collision	Use namespaces: /scan (real) vs /sim/scan (Gazebo)
Gazebo Classic confusion	This course uses modern Gazebo Sim — not Gazebo Classic (EOL Jan 2025)

TF debugging: most navigation problems become frame problems

TF tells ROS where frames are relative to each other in space and time.



What to check when RViz says "No transform"

- Is the Fixed Frame correct? (map vs odom vs base_link)
- Is /tf publishing? → ros2 topic echo /tf
- Are timestamps reasonable? Is use_sim_time consistent across all nodes?
- Is base_link connected to laser in the TF tree?

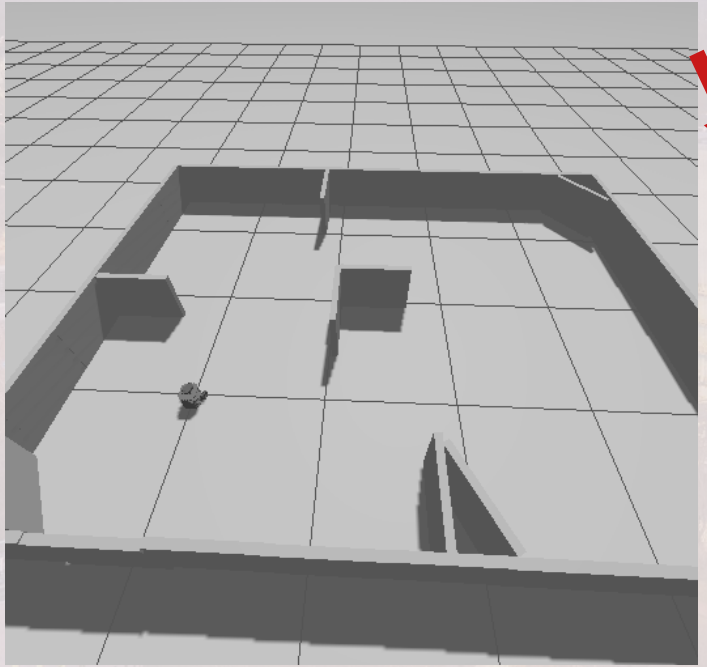
RViz2 as a diagnostic cockpit

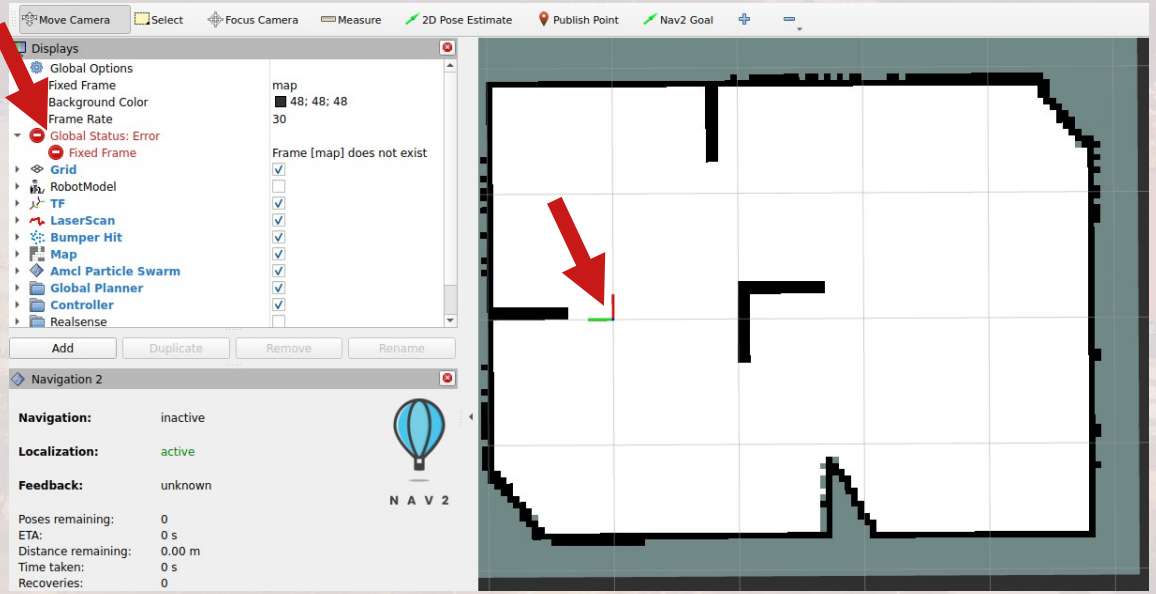
RViz is not only for showing demos: it is a primary troubleshooting tool.

Fixed Frame map or odom depending on the system state	**RobotModel** Is the URDF loaded and correctly transformed?	**LaserScan** Does /scan appear directly in front of the robot?
Odometry Does the pose arrow move when the robot moves?	**TF display** Are all frames connected and stable?	**Map / Costmap** Does Nav2 understand the current world?

RViz2 as a diagnostic cockpit

Necessary commands can be found here!



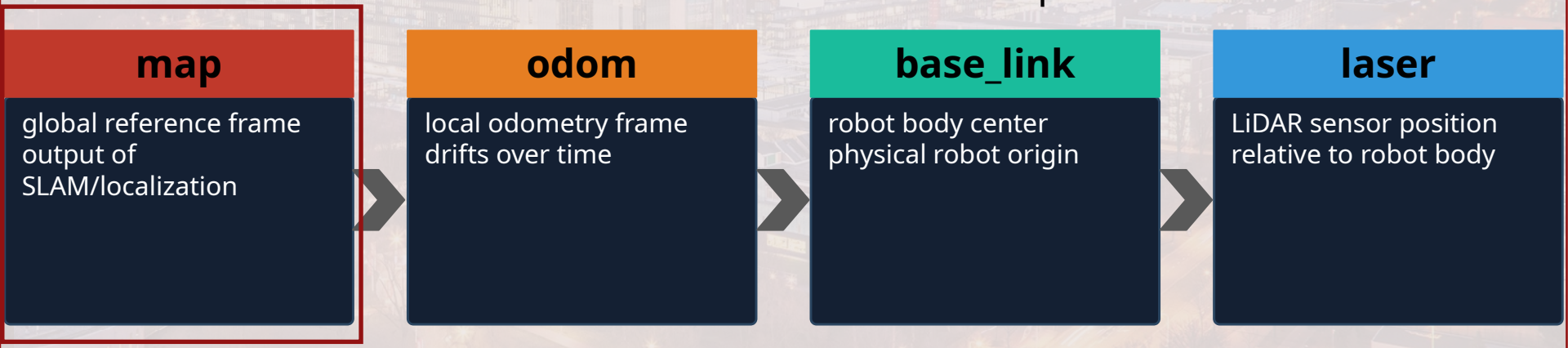



```
[rviz2-2] [INFO] [1779984815.307160640] [rviz2]: Message Filter dropping message: frame 'odom' at time 87.020 for reason
 'discarding message because the queue is full'
[rviz2-2] [INFO] [1779984815.338954767] [rviz2]: Message Filter dropping message: frame 'base_scan' at time 86.200 for r
eason 'discarding message because the queue is full'
[rviz2-2] [INFO] [1779984815.498992409] [rviz2]: Message Filter dropping message: frame 'odom' at time 87.160 for reason
 'discarding message because the queue is full'
[component_container_isolated-1] [INFO] [1779984815.601466529] [global_costmap.global_costmap]: Timed out waiting for tr
ansform from base_link to map to become available, tf error  Invalid frame ID "map" passed to canTransform argument targ
et_frame - frame does not exist
```

TU/e

TF debugging: most navigation problems become frame problems

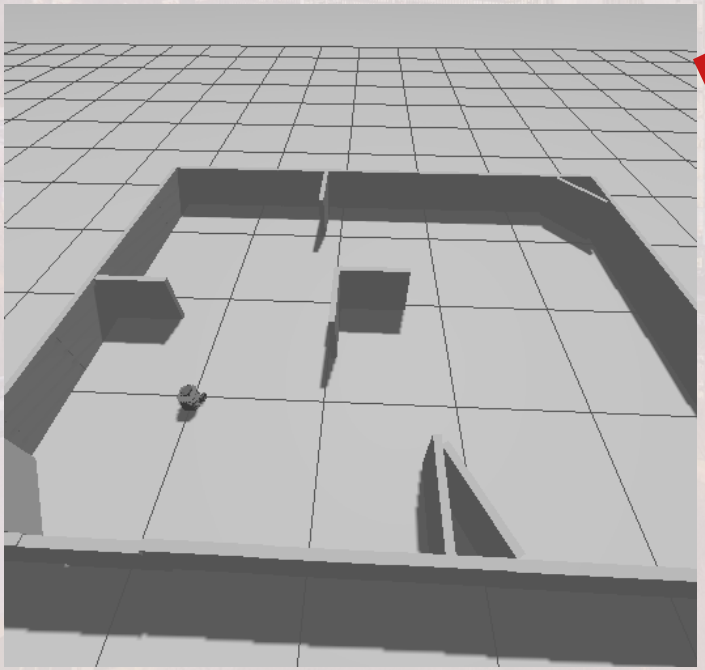
TF tells ROS where frames are relative to each other in space and time.

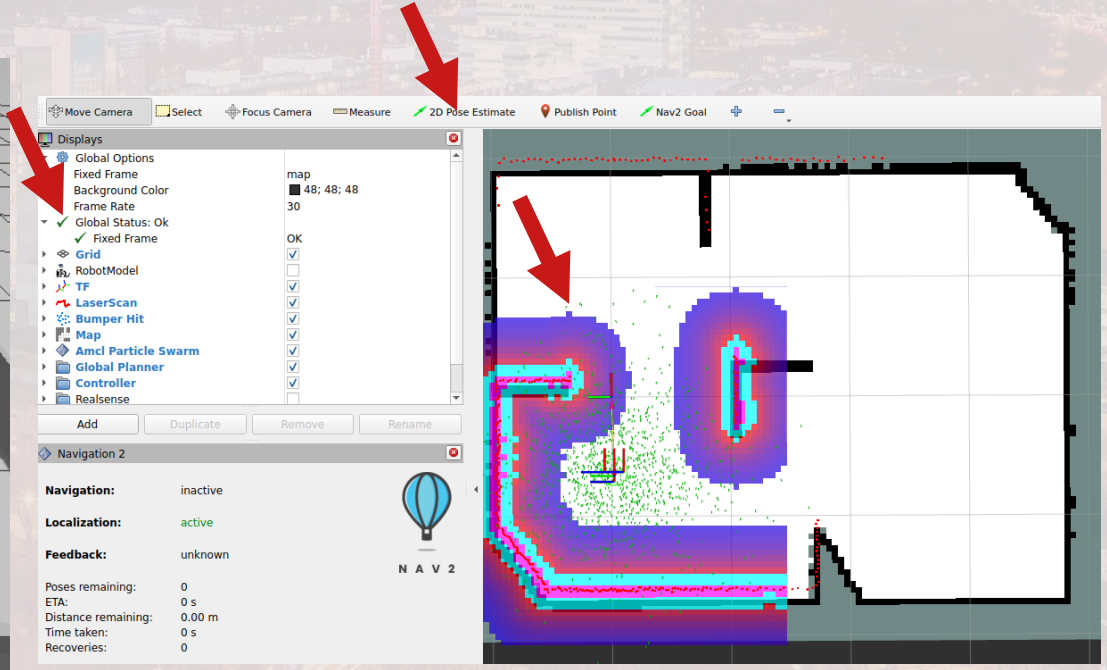


What to check when RViz says "No transform"

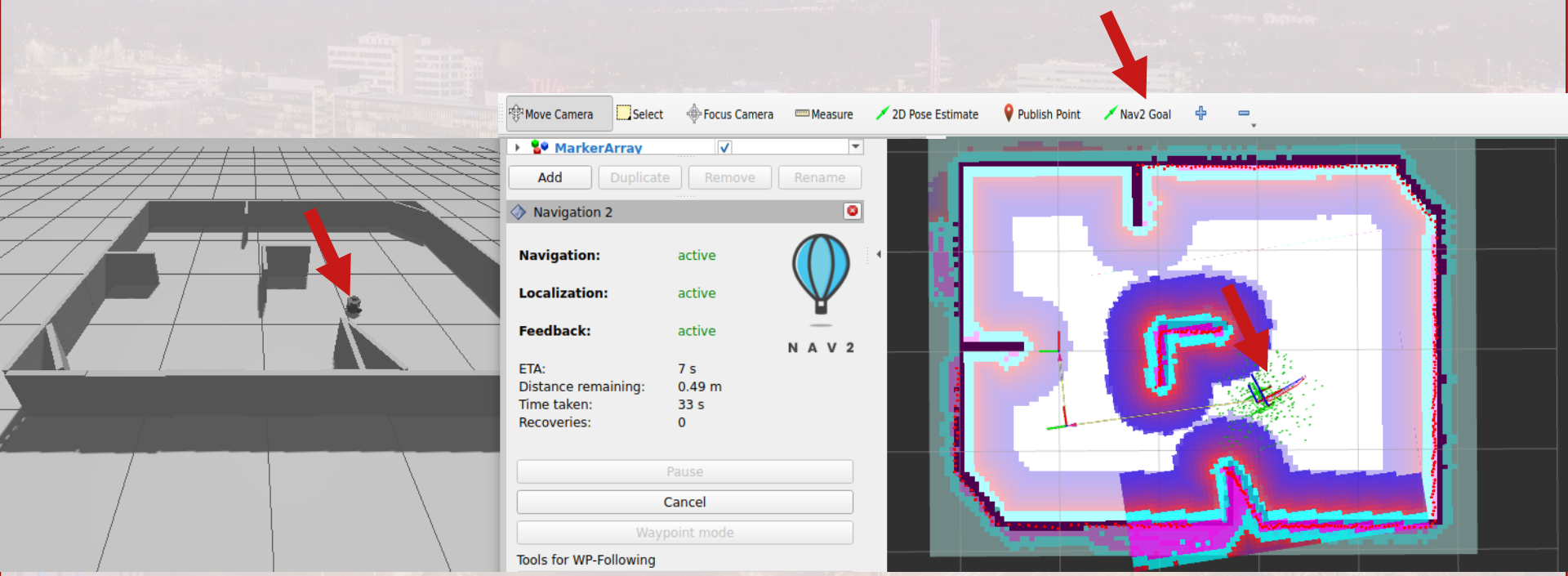
- Is the Fixed Frame correct? (map vs odom vs base_link)
- Is /tf publishing? → ros2 topic echo /tf
- Are timestamps reasonable? Is use_sim_time consistent across all nodes?
- Is base_link connected to laser in the TF tree?

RViz2 as a diagnostic cockpit





RViz2 as a diagnostic cockpit



Network and multi-machine checks

The robot, laptop, and simulation must discover each other safely and consistently.

Symptoms

- Robot topics do not appear on the laptop
- Teleop publishes, but robot does not move
- Another group's robot appears in your graph
- Real and sim robot share the same topic names
- Discovery works once, then disappears

Checks

- Correct Wi-Fi SSID and IP address?
- Robot bringup running? Correct ROS_DOMAIN_ID?
- Do not change lab network settings
- Use namespaces and explicit remapping
- Run ros2 topic list before launching any controller

Troubleshooting should primarily be carried out outside of lab sessions.
If it must occur during a lab session, please prepare a clear and concrete plan in advance.

Case study: Gazebo robot moves, RViz shows no scan

Debug from ROS graph → message stream → TF → RViz display.

1. **Does /scan exist?**
   `ros2 topic list`

2. **Who publishes and subscribes?**
   `ros2 topic info /scan -v`

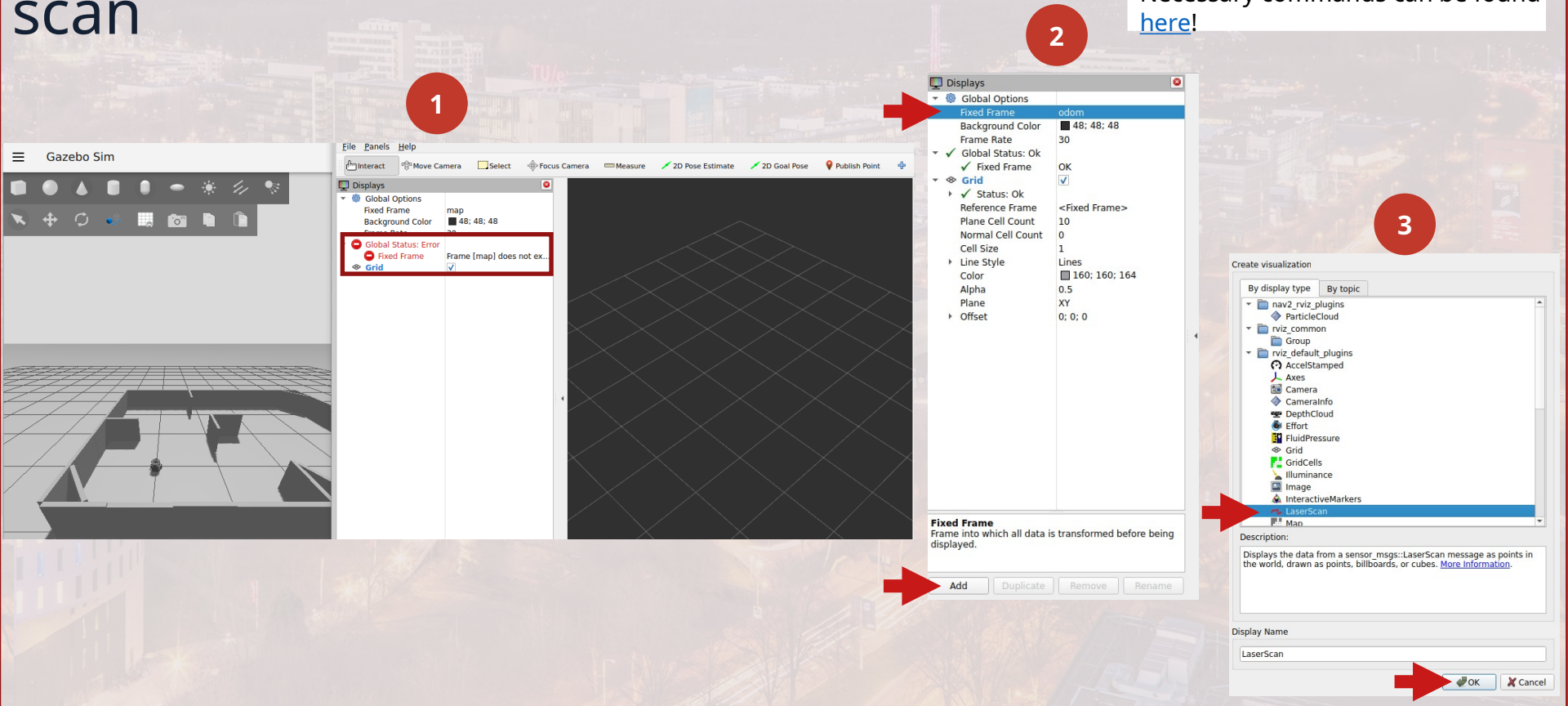
3. **Is data actually arriving?**
   `ros2 topic echo /scan`

4. **Does laser → base_link exist?**
   `ros2 run tf2_tools view_frames`

5. **Correct topic name and fixed frame set?**
   `RViz LaserScan display`

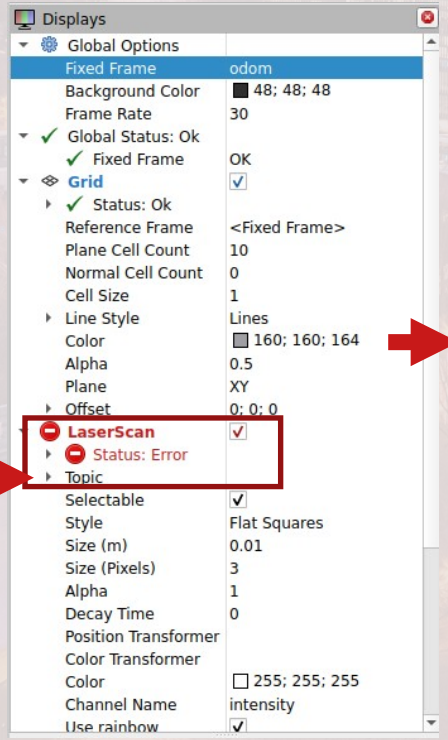
Case study: Gazebo robot moves, RViz shows no scan

Necessary commands can be found here!

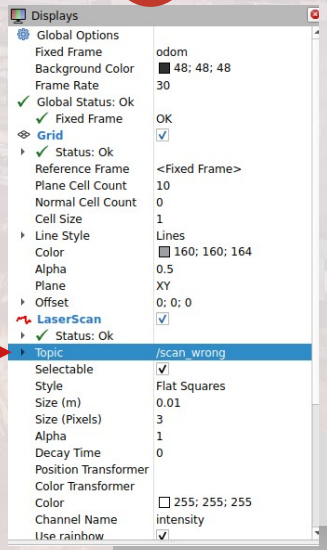


Case study: Gazebo robot moves, RViz shows no scan

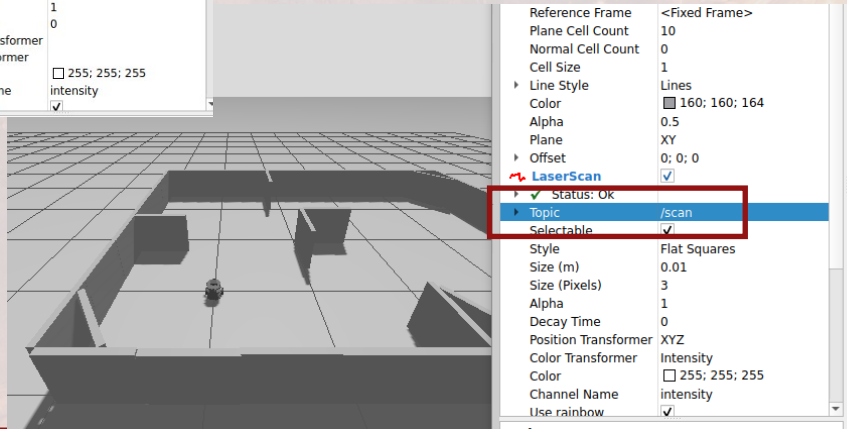
4

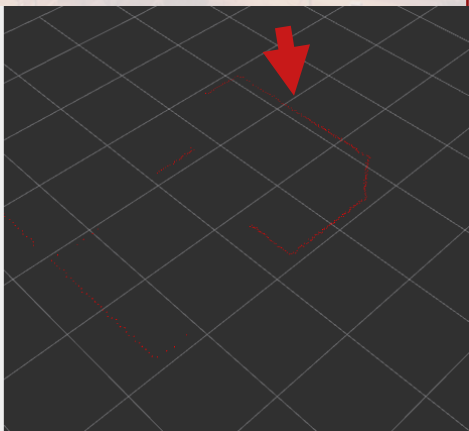


5



6





Recap

Debugging a twin is a structured evidence process.

ROS Graph
nodes, topics, services, actions
CLI proves connections exist

TF
map → odom → base_link → laser
RViz proves spatial meaning

RViz2
scan, odom, TF, robot model
Visualization detects mismatch

Gazebo
model, world, bridge, clock
Simulation reproduces safely

Hardware
battery, LiDAR, OpenCR, network
Lab checks protect scarce time

Troubleshoot in layers: first prove data exists, then prove it arrives at the right rate, then prove it is spatially and behaviorally meaningful.

Mind map: Fault Domain → Diagnostic Tool → CLI Command → Evidence

TU/e

Learning outcome!

By the end of this lecture, you should know how to diagnose failures syste

1. **Locate the fault domain:**
   hardware, ROS graph, TF, RViz, Gazebo, network
2. **Use ROS 2 CLI tools:**
   node / topic / service / action / param inspection
3. **Debug twin-specific failures:**
   bidirectional flow, state sync, latency, environment events
4. **Twin system becomes distributed and robust:**
   Launch files, TF2, Nav2, QoS, rosbag2
5. **Prepare for the final lab review in week 8:**
   evidence: logs, topic echoes, screenshots, demos

Robust system

ROS 2 features that matter when a twin becomes distributed and robust

Launch Files & Parameters
orchestrate multiple nodes; tune behavior without rewriting code

TF2 Transform System
track coordinate frames over time; essential for SLAM and navigation

SLAM & Nav2 Overview
build maps from LiDAR; connect to the same stack on real and sim robot

rosbag2 & Data Replay
record once, replay in simulation; collect evidence for twin verification

QoS Policies
choose reliability and history behavior for commands vs sensors

Launch files and parameter management

Launch files orchestrate your entire system; parameters let you tune it without rewriting code.

Launch Files (.launch.py)

- Start multiple nodes with one command
- Pass parameters, remappings, arguments
- Composable with IncludeLaunchDescription
- Critical for repeatable lab sessions

Example (Python):

```
from launch import LaunchDescription
from launch_ros.actions import Node
def generate_launch_description():
  return LaunchDescription([
    Node(package='tb3_bringup',
         executable='bringup_node')])
```

Parameters (node settings)

- Tune behavior without changing code
- Set via YAML files, CLI, or launch args
- use_sim_time is a critical parameter

CLI examples:

```
ros2 param list /my_node
ros2 param get /my_node stop_dist
ros2 param set /my_node stop_dist 0.5
```

YAML example:

```
my_node:
  ros__parameters:
    stop_distance: 0.5
    use_sim_time: true
```

TF2 — the transform system

Look at to the slide 16!

TF2 tracks where every frame is relative to every other frame, over time.

map
global reference frame
output of SLAM/localization

odom
local odometry frame
drifts over time

base_link
robot body center
physical robot origin

laser
LiDAR sensor position
relative to robot body

What TF2 enables

- Transforms sensor data into robot frame for SLAM
- Allows RViz to show everything in a common coordinate space
- Required for Nav2 navigation to work correctly

CLI and debugging

```
ros2 run tf2_tools view_frames
ros2 topic echo /tf
rviz2 → TF display panel
```

TU/e

SLAM and Nav2 overview

Run the same ROS 2 navigation stack against Gazebo Sim and the physical TurtleBot.

SLAM Toolbox

- Builds a 2D occupancy grid map from LiDAR + odometry
- Creates the map → odom → base_link TF chain
- Online mode: build map in real-time
- Localization mode: use a saved map

```
Key topic: /map (nav_msgs/OccupancyGrid)
Launch: slam_toolbox online_sync_launch.py
```

Nav2 Stack

- Global planner: finds a path to the goal
- Local planner / controller: follows path reactively
- Costmaps: inflation of obstacles for safe clearance
- Recovery behaviors: spin, back-up, wait

```
Key action: /navigate_to_pose
Launch: navigation_launch.py (with map)
```

DT relevance: the same SLAM and Nav2 launch files run against Gazebo during development, then against the physical Turtlebot for validation.

QoS: the hidden contract for message delivery

Quality of Service controls how ROS 2 transports messages under real network conditions.

Reliability

BEST_EFFORT for fast sensors (dropped messages ok)
RELIABLE for critical state or commands

History / Depth

How many old messages are kept if subscriber is late
(KEEP_LAST N or KEEP_ALL)

Durability

Can a late subscriber receive the last published value?
(VOLATILE vs TRANSIENT_LOCAL)

Deadline / Liveliness

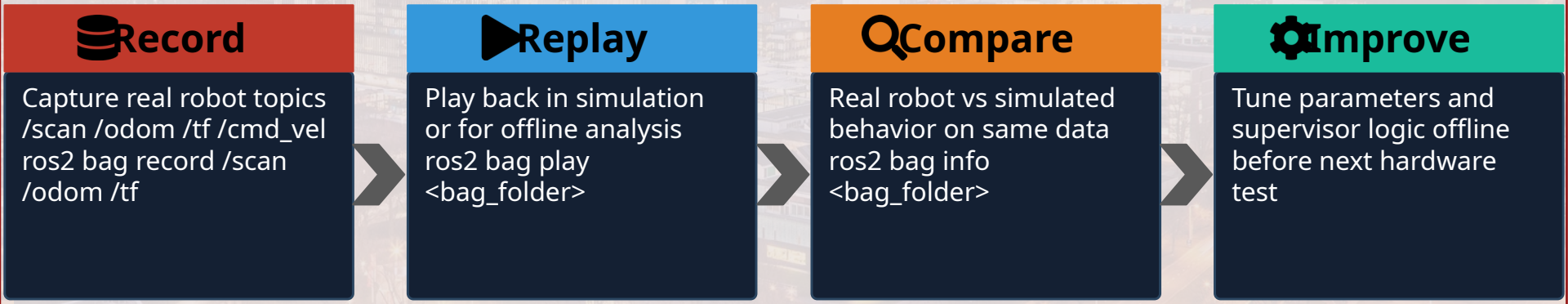
Can the system detect missing or stale publishers?
Useful for fault detection in twin systems

Teaching example: /scan can tolerate dropped samples (BEST_EFFORT) but /mode, /battery_state, or /fault_state should use RELIABLE - mismatched QoS means silent data loss.

TU/e

rosbag2: record once, debug many times

For digital twins, data replay is a powerful testing and evidence collection tool.



DT relevance: record the physical TurtleBot → replay in Gazebo → compare behavior → this is exactly the periodic twin data sync loop.

```
CLI: ros2 bag record /scan /odom /tf   |   ros2 bag play <bag_folder>   |   ros2 bag info <bag_folder>
```

DDS and distributed discovery

ROS 2 does not rely on a ROS 1-style master – nodes discover each other through DDS.

Robot SBC

real robot nodes
LiDAR, OpenCR, bringup

Lab Laptop

teleop, RViz2,
DT supervisor node

Gazebo PC / Container

simulation + ros_gz
bridge nodes

This is powerful for twins – but it means ROS_DOMAIN_ID, network settings, and QoS all matter for correct operation.

ROS 1
Centralized ROS Master
Single point of failure
No built-in QoS

ROS 2
DDS distributed discovery
No central master
QoS policies built-in

Putting advanced topics together: a robust twin supervisor

The advanced tools each support the same three course requirements.

Inputs

```
/scan_real
/scan_sim
/odom
/battery_state
/weather
/fault_state
```

Twin Supervisor

- QoS choices per topic
- Parameters for thresholds
- Actions for tasks
- rosbag2 for logging
- TF2 for frame consistency
- Launch file to start all

Outputs

```
/cmd_vel_real
/cmd_vel_sim
/mode
/alerts
/sync_status
```

Robustness is not one feature: it is the combination of interface design, QoS, monitoring, data replay, and safe defaults – all tied together by a launch file.

Recap

Advanced ROS 2 topics help when your twin becomes distributed, live, and safety-sensitive.

Launch + Params
orchestrate & tune behavior

TF2
coordinate frames over time

SLAM / Nav2
mapping & autonomous navigation

QoS
delivery contract per topic

rosbag2
record / replay evidence

Takeaway: choose the ROS 2 mechanism that matches the behavior you need – continuous stream, one request, long task, tunable setting, or replayable evidence.

Mind map: Advanced ROS 2 → Launch → TF2 → SLAM/Nav2 → QoS → rosbag2 → DT Integration

Learning outcome!

By the end of this lecture, you should know how to diagnose failures syste

1. **Locate the fault domain:**
   hardware, ROS graph, TF, RViz, Gazebo, network
2. **Use ROS 2 CLI tools:**
   node / topic / service / action / param inspection
3. **Debug twin-specific failures:**
   bidirectional flow, state sync, latency, environment events
4. **Twin system becomes distributed and robust:**
   Launch files, TF2, Nav2, QoS, rosbag2
5. **Prepare for the final lab review in week 8:**
   evidence: logs, topic echoes, screenshots, demos

Evidence for Week 4/5 and Week 8 technical reviews

A good review shows proof, not plans. Collect evidence from day one.

Week 4/5 baseline

- Gazebo launches and robot is visible
- Simulated robot moves with teleop
- Workspace builds without major errors
- Physical robot: short safe teleop test done
- One topic each direction identified
- One state + one interaction chosen

Week 8 verification

- Bidirectional pub/sub shown (topic list + echo)
- (Non-)motion state sync demonstrated
- Environment/object interaction shown
- Evidence: logs, ros2 topic echo, screenshots
- Demo video story prepared
- Latency measured and documented